

Scalability, Availability & Consistency

Why this chapter matters: Chapters 1–4 explained one machine and how machines talk. From here we build **systems of many machines**. Three questions run through every design interview: **Can it grow?** (scalability) **Does it stay up?** (availability) **Do all users see the right data?** (consistency). They pull against each other, and senior engineers are paid to choose the right trade-off.

🔗 QUICK RECALL — Chapter map

- 5.1 What is scalability → 5.2 Vertical vs horizontal, statelessness, scale cube → 5.3 The scaling journey
- 5.4 Availability: nines, series/parallel math, MTBF/MTTR, SLI/SLO/SLA → 5.5 Availability patterns: failover, replication, multi-AZ, DR
- 5.6 CAP theorem & PACELC → 5.7 Consistency models, quorums, conflict resolution → 5.8 Choosing trade-offs by use case
- 5.9 Practice, Q&A, cheat sheet

5.1 What is scalability?

Scalability is a system's ability to keep acceptable performance as **load grows** by adding resources — ideally cost growing roughly in line with load, without redesigning from scratch. Load can grow in several dimensions: more users, more requests per second, more data, more geography, even more engineers working on the code.

	Performance problem	Scalability problem
Symptom	The system is slow for a single user	Fast for one user, but slow or failing under heavy load
Typical fix	Optimise code, indexes, queries, caching	Add capacity: more servers, partition data, remove shared bottlenecks
Measure with	Latency of one request	Throughput & latency percentiles as load increases

Describe load with numbers (“load parameters”)

- Requests/second (average **and peak**), read:write ratio, concurrent users/connections, data volume and growth per day, cache hit ratio, fan-out (one post → N followers).
- Describe performance with **percentiles**: p50 (typical), p95, p99, p99.9 — averages hide the slow tail that users actually notice (Chapter 2).

◆ SENIOR INSIGHT — Linear scalability and what breaks it

- Ideal: double the servers → double the throughput. Real systems lose efficiency to **serial parts** (Amdahl's law, Chapter 1), **coordination** (locks, consensus, cross-shard transactions), **shared resources** (one database, one queue, one cache node) and **hot spots** (a celebrity user, a viral item).
- Scaling means finding the current bottleneck, removing it, and then finding the next one. **Measure first**; scaling the wrong tier wastes money.

5.2 Vertical vs horizontal scaling

	Vertical (scale up)	Horizontal (scale out)
What	Bigger machine: more CPU, RAM, faster disks	More machines behind a load balancer / partitioned data
Pros	Simple — no code changes; no distributed-systems problems; strong consistency is easy	No hard ceiling; cheaper commodity hardware; built-in redundancy (lose one node, keep serving); scale in small steps
Cons	Hardware ceiling ; cost rises steeply (expensive specialised servers); still a single point of failure ; upgrade often means downtime	Complexity: load balancing, data partitioning, consistency, distributed debugging; more network hops; state must be externalised
Typical use	Databases (first step), small systems, quick relief	Stateless web/app tiers, caches, queues, sharded stores

★ MUST REMEMBER — Real systems do both

- Scale a **stateless tier horizontally** (easy), scale the **database vertically first** (simple), then add replicas and finally sharding. Horizontal scaling is also an **availability** tool: N small servers survive failures better than one big one.

Statelessness — the key that unlocks horizontal scaling

A server is **stateless** if it keeps no user-specific data between requests, so **any** instance can handle **any** request. Then adding or removing servers is trivial.

State that creeps in	Where to put it instead
User sessions in server memory	Shared store (Redis/Memcached/DB) or signed tokens (JWT) in the client
Uploaded files / profile pictures on local disk	Object storage (S3) + CDN (Chapter 2)
Local in-memory cache with per-server state	Distributed cache, or accept short-lived duplicates
Scheduled jobs running on every instance	A single scheduler / leader election, or a job queue
WebSocket connections (inherently stateful)	Connection registry + pub/sub across gateway servers (Chapter 4)

◆ SENIOR INSIGHT — Pets vs cattle & shared-nothing

- Treat servers as **cattle** (identical, disposable, replaced automatically by autoscaling/Kubernetes), not **pets** (hand-configured, irreplaceable).
- Shared-nothing architecture**: nodes share no memory or disk, only talk over the network — the model behind most scalable data stores. Scaling then depends on how you **partition the data**.

The Scale Cube (three directions to scale)

Axis	Technique	Example	Solves	Costs
X	Clone identical instances behind a load balancer	10 copies of the API server	Throughput, availability	Doesn't help data size or code complexity
Y	Split by function (services / microservices)	Users, Orders, Search as separate services	Code complexity, team scale, independent scaling	Distributed-systems complexity (Chapter 8)
Z	Split by data (partition/shard by key)	Users A-M on shard 1, N-Z on shard 2; or by user_id hash	Data size, write throughput	Cross-shard queries, rebalancing (Chapter 9)

5.3 The scaling journey (from one box to millions of users)

Almost every system design answer follows this evolution. Each step is triggered by a **bottleneck** and creates a **new problem** that the next step solves. Memorise the pattern; the chapters ahead detail each step.

#	Stage	Trigger (bottleneck)	Solution	New problem created
1	Single server: web + app + DB on one box	Start	—	SPOF; components fight for CPU/RAM
2	Separate the database from the app	Resource contention	App server + DB server (or managed DB)	Network hop; DB is still a SPOF
3	Vertical scale	Quick relief	Bigger instance	Ceiling, cost, downtime
4	Load balancer + several stateless app servers	App CPU saturated; need HA	Horizontal scale; sessions → Redis	LB itself is a SPOF; DB becomes the bottleneck
5	Cache + CDN	DB read load, latency	Cache-aside (Redis), CDN for static (Ch. 6, 11)	Invalidation, stale data
6	Read replicas	Read-heavy DB	Leader-follower replication (Ch. 9)	Replication lag (eventual consistency)
7	Async processing	Slow requests, traffic spikes	Message queue + workers (Ch. 12)	Eventual results, retries, idempotency
8	Shard / partition the data or NoSQL	Write throughput, data size	Sharding by key (Ch. 9–10)	Cross-shard joins/transactions, hot keys
9	Multi-AZ → multi-region	Latency to distant users, DR	Geo-replication, geo-DNS	Cross-region consistency, cost
10	Split into services	Team/org scale	Microservices (Ch. 8)	Network failures, observability, versioning

✗ COMMON MISTAKE — Don't start at step 10

- Premature distribution is a classic mistake: a well-indexed single PostgreSQL with a cache serves a surprising amount of traffic. In interviews, **start simple, estimate the load (Chapter 2), and add complexity only where numbers demand it.**

5.4 Availability

Availability is the fraction of time a service is working and able to serve requests: **Availability = Uptime / (Uptime + Downtime)**. It is usually quoted in “nines”.

The nines table

Availability	Nines	Downtime / year	Downtime / month	Downtime / week	Downtime / day
99%	two 9s	3.65 days	7 h 18 min	1 h 41 min	14 min 24 s
99.9%	three 9s	8 h 45 min 58 s	43 min 50 s	10 min 5 s	1 min 26 s
99.95%		4 h 23 min	21 min 55 s	5 min 2 s	43 s
99.99%	four 9s	52 min 36 s	4 min 23 s	1 min (60.5 s)	8.6 s
99.999%	five 9s	5 min 16 s	26 s	6 s	0.9 s
99.9999%	six 9s	31.6 s	2.6 s	0.6 s	0.09 s

✎ **CORRECTION TO YOUR ORIGINAL NOTE** — Typo in the commonly-copied availability table

- The popular table (also in your notes) lists “1m 5s” downtime per week for four 9s. Correct value: 0.01% of 604,800 s = **60.5 s ≈ 1 min**. The other figures in your notes are correct. Tip: **each extra nine cuts allowed downtime by 10x** — and costs far more than 10x to achieve.

★ **MUST REMEMBER** — Memorise three anchors

- **99.9% ≈ 8.8 hours/year ≈ 43 min/month** • **99.99% ≈ 52 min/year ≈ 4.4 min/month** • **99.999% ≈ 5 min/year**.

Availability in series vs in parallel

If a service consists of multiple components prone to failure, overall availability depends on how they are connected.

	In sequence (all must work)	In parallel (any one is enough)
Formula	$A_{\text{total}} = A_1 \times A_2 \times \dots \times A_n$	$A_{\text{total}} = 1 - (1 - A_1) \times (1 - A_2) \times \dots$
Effect	Availability decreases (weaker than the weakest link)	Availability increases (redundancy)
Example: two 99.9% components	$0.999 \times 0.999 = \mathbf{99.8\%}$	$1 - 0.001 \times 0.001 = \mathbf{99.9999\%}$
Real-world picture	A request chain: LB → app → cache → DB → payment API	Two app servers behind a LB, DB primary + standby

Worked example (three-tier system): load balancer 99.99% (redundant pair), app tier = 2 servers in parallel of 99.9% each = 99.9999%, database 99.95%.

Total = $0.9999 \times 0.999999 \times 0.9995 = 0.9994$ → about 99.94% (~5.3 hours downtime/year)
The database is the weakest link → that is where to add redundancy (standby, replicas, multi-AZ).

◆ **SENIOR INSIGHT** — Why the math flatters you — and a chain of microservices is dangerous

- **Series trap:** 10 services each 99.9% in a synchronous call chain = $0.999^{10} \approx \mathbf{99.0\%}$ (≈ 3.7 days of downtime/year). Mitigate with fewer hard dependencies, timeouts, fallbacks, caching, async decoupling.
- **Parallel assumption:** the formula assumes **independent** failures. Replicas in the same rack/AZ, sharing a deployment, a config push, a library bug or one overloaded dependency fail **together** (correlated failures). Real availability is lower than the formula; spread across failure domains and stagger rollouts.
- The **failover mechanism itself** (health checks, load balancer, DNS) must be at least as reliable as what it protects.

MTBF, MTTR and what really improves availability

Availability = $\text{MTBF} / (\text{MTBF} + \text{MTTR})$
MTBF = Mean Time Between Failures MTTR = Mean Time To Recover/Repair
Example: MTBF 1000 h, MTTR 1 h → $1000/1001 = 99.9\%$ MTTR 6 min → 99.99%

You can raise availability by failing **less often** (quality, testing, redundancy) or, often cheaper and more realistic, by **recovering faster** (automatic failover, health checks, fast rollbacks, runbooks, good alerting). Modern SRE practice emphasises **low MTTR**.

SLI, SLO, SLA and error budgets

Term	Meaning	Example
SLI — Service Level Indicator	A measured metric of service health	% of requests answered successfully in < 300 ms
SLO — Service Level Objective	Internal target for an SLI over a window	99.9% of requests succeed, measured over 30 days
SLA — Service Level Agreement	Contract with customers, with penalties / credits; usually looser than the SLO	99.5% monthly uptime or service credits
Error budget	Allowed unreliability = 1 – SLO	99.9% → 0.1% of 30 days = 43.2 minutes of failure per month

◆ SENIOR INSIGHT — Error budgets turn reliability into a decision tool

- Budget left → ship features faster. Budget spent → freeze risky releases and invest in reliability. This aligns product and operations teams around a number instead of opinions.
- Choose SLOs from user pain, not from what you can currently achieve; 100% is the wrong target (cost explodes and change stops).

★ MUST REMEMBER — Availability vs reliability vs durability

- **Availability**: can I use it right now? **Reliability**: does it behave correctly over time? **Durability**: is stored data safe from loss (e.g., S3 advertises 11 nines of durability while availability is ~99.99%). A system can be available but wrong, or durable but temporarily unreachable.

5.5 Availability patterns

Two complementary patterns give high availability: **fail-over** (switch to a spare when the active fails) and **replication** (keep copies of data on several nodes). Underneath both sits **redundancy** with no single point of failure (SPOF).

Fail-over

	Active-passive (master-slave / primary-standby)	Active-active (master-master)
How	Heartbeats flow between active and passive. If the heartbeat stops, the passive takes over the active's IP (virtual/floating IP) and resumes service. Only the active handles traffic.	Both servers handle traffic , spreading load between them
Downtime	Depends on standby type: hot (running, in sync → seconds), warm (running, behind), cold (must be started → minutes)	Minimal: the other node already serves traffic
Routing needs	VIP / DNS update / LB health check	Public-facing: DNS or LB must know both public IPs. Internal: application logic must know both servers
Cost	Standby capacity sits idle	Better utilisation, but each node must handle the full load if the other fails (N+1 sizing)
Hard part	Failure detection and promotion	Keeping state consistent in two writers (conflicts)

Disadvantages of fail-over: more hardware and complexity; potential **data loss** if the active fails before newly written data was replicated to the passive (RPO > 0). Modern terminology prefers **primary/replica** or **leader/follower** over master/slave.

◆ SENIOR INSIGHT — Split brain — the scariest failover bug

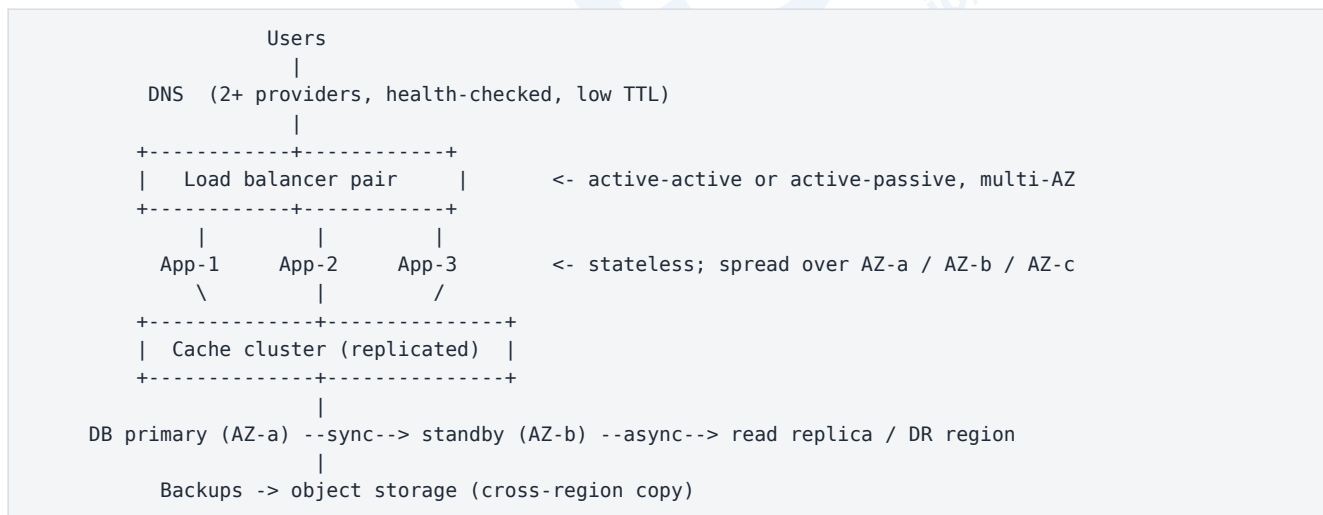
- If a network partition makes the standby believe the primary is dead, **both** may act as primary and accept writes → divergent data. Prevent with **quorum / majority voting** (need $> N/2$ votes to be leader), **fencing tokens** and **STONITH** (shoot the other node in the head: power it off), and consensus systems like etcd/ZooKeeper (Raft/Paxos, Chapter 14).
- Failure detection is imperfect: a slow node looks like a dead node (**gray failure**). Tune heartbeat timeouts: too short → false failovers, too long → long outages.

Replication (preview of Chapter 9)

Mode	Write is acknowledged when...	Pros	Cons
Synchronous	replica(s) have also written it	No data loss on failover (RPO = 0); replicas up-to-date	Higher write latency; if the replica is down or slow, writes stall
Asynchronous	only the leader has written it	Fast writes; leader unaffected by slow replicas	Replication lag; data loss if leader dies before shipping (RPO > 0)
Semi-synchronous	leader + at least one replica	Balance: durable copy exists elsewhere	Slightly slower; some complexity

Topologies: **leader-follower** (one writer, many readers), **multi-leader** (several writers, conflict resolution needed), **leaderless** (any node accepts writes; quorums, as in Dynamo/Cassandra — see 5.7).

Putting it together: a highly available architecture



Failure domains

Domain	Failure example	Protection
Process / host	Crash, OOM, hardware fault	Multiple instances, autoscaling group, auto-restart
Rack / power / switch	Top-of-rack switch dies	Spread across racks
Availability Zone (AZ)	Data-centre fire, power or network outage	Deploy across ≥ 2 -3 AZs (multi-AZ)
Region	Large-scale cloud incident, disaster	Multi-region: warm standby or active-active; DNS/geo routing
Software / config	Bad deploy, bug, expired certificate	Canary + gradual rollout, feature flags, fast rollback, circuit breakers
Dependency	Third-party API or DNS provider down	Timeouts, fallbacks, caching, multiple providers

Disaster-recovery strategies (cost vs recovery time)

Strategy	What runs in the DR site	RTO	RPO	Cost
Backup & restore	Nothing; restore from backups	Hours–days	Hours	\$
Pilot light	Core data replicated; app servers off	Tens of minutes–hours	Minutes	\$\$
Warm standby	Scaled-down full copy running	Minutes	Seconds–minutes	\$\$\$
Multi-site active–active	Full capacity in 2+ regions serving traffic	≈ 0–seconds	≈ 0	\$\$\$\$

◆ SENIOR INSIGHT — Design for failure — habits of highly available systems

- **Graceful degradation:** if recommendations are down, still show the page. **Load shedding:** reject excess requests early (503) rather than collapse. **Timeouts, retries with jitter, circuit breakers, bulkheads** (Chapter 4).
- **Health checks** (shallow for the LB, deep for monitoring), **autoscaling + self-healing, canary releases, chaos engineering** (deliberately kill instances to prove you survive it), and regular **failover drills** — an untested failover plan is not a plan.
- Remember Chapter 2: **RAID/replication is not a backup;** keep versioned, off-site backups and test restores.

5.6 CAP theorem

In a distributed data system, three properties are desirable:

Property	Precise meaning	Careful!
Consistency	Every read receives the most recent write or an error (all nodes appear to hold one up-to-date copy — linearizability)	Not the same as the “C” in ACID (valid state). Here it means all clients see the same data at the same time
Availability	Every request to a non-failing node receives a (non-error) response — without a guarantee that it contains the most recent write	Not “99.99% uptime”; it means each live node must answer
Partition tolerance	The system keeps operating despite arbitrary message loss or delay between nodes (network partition)	Partitions happen in real networks; you can’t opt out

★ MUST REMEMBER — The correct statement of CAP

- **Networks aren’t reliable, so you must tolerate partitions (P). During a partition you must choose between Consistency and Availability.** When there is no partition you can have both.
- That’s why “pick any two of three” is misleading: **CA** only exists for a single node (no network to partition) — it isn’t a real distributed option.

What happens during a partition

```
Client A --write x=5--> [ Node N1 ] --X-- network partition --X-- [ Node N2 ] <--read x-- Client B
```

CP choice: N2 cannot confirm it has the latest value -> returns an ERROR / timeout
(consistent, but NOT available on the minority side)

AP choice: N2 answers with what it has (x = 4, stale) -> every request gets a response
(available, but NOT consistent); the nodes reconcile after the partition heals

	CP — consistency + partition tolerance	AP — availability + partition tolerance
Behaviour in a partition	Refuses or times out requests it can’t guarantee correct (waiting for the partitioned node may cause a timeout error)	Serves the most readily available (possibly stale) version from any node; writes propagate after the partition heals

	CP — consistency + partition tolerance	AP — availability + partition tolerance
Good when	The business needs atomic, correct reads and writes : money, inventory counts, locks, leader election, config	The business can allow eventual consistency or must keep working despite external errors: carts, feeds, likes, sessions, DNS
Typical systems*	ZooKeeper, etcd, HBase, Spanner, MongoDB (majority writes), single-leader SQL with sync replication	Cassandra, DynamoDB, Riak, CouchDB, DNS

* Many systems are **tunable** per request (e.g., Cassandra QUORUM vs ONE, DynamoDB strongly consistent reads, MongoDB read/write concerns), so labels are tendencies, not absolutes.

◆ SENIOR INSIGHT — PACELC — what CAP leaves out

- CAP only talks about partitions, but partitions are rare. Daniel Abadi's **PACELC**: if there is a **P**artition, choose **A** or **C**; **E**lse (normal operation), choose **L**atency or **C**onsistency. Even without failures, synchronous replication costs latency.
- Commonly cited classifications: Cassandra/Dynamo/Riak = **PA/EL** (fast and available); MongoDB (default) = **PA/EC**; HBase/Spanner = **PC/EC** (consistent always; Spanner uses atomic clocks to keep latency tolerable).
- Takeaway: consistency is **always** bought with latency (coordination across replicas), and cross-region consistency costs tens to hundreds of milliseconds (speed of light, Chapter 3).

🔗 CORRECTION TO YOUR ORIGINAL NOTE — Refinement of your CAP notes

- Your notes say you “can only support two of the three”. The practical rule is: **P is mandatory in a distributed system, so the real choice is C vs A while a partition lasts**. Also the C in CAP is linearizability, not ACID consistency.

5.7 Consistency patterns (models)

With several copies of the same data, we must decide how to synchronise them so clients get a consistent view. Recall the CAP definition: *every read receives the most recent write or an error*. Models form a spectrum from strict to relaxed — stronger means simpler for developers but slower and less available.

Model	Guarantee	Typical mechanism	Seen in	Fits
Strong (linearizable)	After a write completes, all reads see it . Behaves like a single copy	Synchronous replication / consensus (Raft, Paxos), single leader reads	File systems, RDBMS, Spanner, etcd	Transactions, payments, inventory, locks
Eventual	If no new writes, all replicas eventually converge (typically ms, but unbounded). Data replicated asynchronously	Async replication, anti-entropy, read repair	DNS, email, Cassandra, DynamoDB, CDN caches	Highly available systems, feeds, counters, catalogues
Weak (best effort)	After a write, reads may or may not see it; lost updates are not recovered	No retry, no replay	memcached, VoIP, video chat, real-time games	Real-time use where stale data is useless: if you lose reception for a few seconds you don't hear what was said

Between strong and eventual: practical session guarantees

Eventual consistency alone surprises users. These cheaper guarantees fix the most visible anomalies:

Guarantee	Problem it prevents	How to implement
Read-your-writes	You post a comment, refresh, and it's missing (read hit a lagging replica)	Read your own recent writes from the leader (e.g., for 10–30 s); or pass a version/timestamp token and wait for replicas to catch up; or sticky routing

Guarantee	Problem it prevents	How to implement
Monotonic reads	You see newer data, then on the next request older data (different replica)	Pin each user/session to one replica (hash of user id)
Consistent prefix	You see an answer before the question (causally related writes seen out of order)	Keep related writes in the same partition / order by causality
Causal consistency	Effects visible before their causes	Vector clocks / dependency tracking; supported by some stores

Quorums — tunable consistency in leaderless systems

With **N** replicas, a write must be acknowledged by **W** replicas and a read must consult **R** replicas. If **R + W > N**, every read set overlaps every write set, so a read sees at least one copy with the latest write.

N = 3	W	R	R + W	Behaviour
Fast, weak	1	1	2 (≤ 3)	Fastest; stale reads possible (eventual)
Balanced (QUORUM)	2	2	4 (> 3)	Overlap guaranteed; tolerates 1 node down for both reads and writes
Read-optimised	3	1	4	Fast reads; writes need every node (low write availability)
Write-optimised	1	3	4	Fast writes; reads slow and fragile

◆ SENIOR INSIGHT — Where quorums and eventual consistency get tricky

- **Conflicts**: two clients write the same key on different replicas. Resolution strategies: **last-write-wins** (simple but can silently lose data and depends on clocks), **vector clocks** to detect concurrent writes and let the app merge, **CRDTs** (data types that merge automatically: counters, sets), or application-level merge (e.g., Amazon cart: union of items).
- **Repair mechanisms** in Dynamo-style stores: **read repair** (fix stale replica when a read notices), **hinted handoff** (hold writes for a down node), **anti-entropy** (Merkle trees compare replicas in the background).
- **Clocks lie**: physical clocks drift, so ordering events by timestamp across machines is unsafe. Use logical/Lamport/vector clocks or hybrid logical clocks; Google Spanner uses TrueTime (GPS + atomic clocks) for global ordering.
- Even $R + W > N$ isn't a full linearizability guarantee in the presence of sloppy quorums, concurrent writes and failures. If you need strict guarantees use consensus or a single-leader design.

5.8 Choosing the trade-off by use case

Use case	Prefer	Reasoning / approach
Bank transfer, payment ledger	CP / strong	Wrong or duplicate money is worse than a failed request: ACID transactions, single leader, sync replication, idempotency keys
Inventory decrement (last item)	Strong on the write path	Atomic UPDATE ... WHERE stock > 0; browsing can be eventually consistent
Shopping cart	AP / eventual	Never block 'add to cart'; merge carts after partitions (Dynamo)
Social feed, likes, view counts	AP / eventual	Slightly stale or approximate is fine; high availability and low latency matter more
User profile edit	Read-your-writes	User must see their own change; others can lag
Leader election, distributed locks, configuration	CP (etcd, ZooKeeper)	Wrong answers (two leaders) are catastrophic; unavailable minority is acceptable
DNS, CDN content	AP / eventual + TTL	Global availability over instant freshness
Chat / messaging	Mostly AP with ordering per conversation	Deliver always, order within a thread, dedupe by message id
Analytics, logs, metrics	Eventual / weak	Throughput matters; occasional loss/delay acceptable

★ MUST REMEMBER — How to answer trade-off questions in an interview

- 1) Ask what is worse for the business: **wrong data** or **no data**? 2) Pick consistency per feature, not per system. 3) Name the mechanism (quorum, leader reads, idempotency, TTL) and the failure it protects against. 4) State the cost (latency, availability, complexity).

5.9 Practice questions (try before looking)

P1. Five services in a synchronous call chain, each 99.9% available. Overall availability and downtime per year?

Answer: $0.999^5 \approx 0.9950 \rightarrow 99.5\% \approx 0.5\% \times 8,766 \text{ h} \approx 43.8 \text{ hours } (\sim 1.8 \text{ days})$ per year.

P2. Two independent replicas, each 99%. Combined availability if either can serve?

Answer: $1 - 0.01 \times 0.01 = 99.99\%$ (assuming independent failures and perfect failover).

P3. Your SLO is 99.95% over 30 days. What is the error budget in minutes?

Answer: $0.05\% \times 43,200 \text{ min} = 21.6 \text{ minutes}$.

P4. MTBF = 500 h, MTTR = 5 h. Availability? What if MTTR drops to 30 minutes?

Answer: $500/505 \approx 99.0\%$; with 0.5 h repair: $500/500.5 \approx 99.9\%$ — faster recovery gave a 10× downtime reduction.

P5. N = 5 replicas, W = 3, R = 3. Is read-after-write guaranteed? How many node failures can writes tolerate?

Answer: $R + W = 6 > 5 \rightarrow$ overlap guaranteed. Writes need 3 of 5 $\rightarrow$ tolerate **2** failures.

P6. During a partition, should a payment service and a shopping-cart service choose C or A?

Answer: Payment: **CP** (reject rather than risk incorrect balances). Cart: **AP** (accept adds, merge later).

Interview Q&A (30-second answers)

Q1. Vertical vs horizontal scaling?

Vertical = bigger machine: simple but capped, costly, and a SPOF. Horizontal = more machines: no hard ceiling and built-in redundancy, but needs stateless services, load balancing and data partitioning.

Q2. What makes a service stateless and why does it matter?

No user-specific state stored on the instance (sessions, files). Any instance can serve any request, so you can add/remove/replace servers freely and fail over without losing sessions.

Q3. Explain CAP in plain words.

In a distributed store, network partitions will happen. While partitioned, you either refuse some requests to keep data correct (CP) or answer from possibly stale data to stay up (AP). With no partition you can have both.

Q4. What is PACELC?

Extension of CAP: if Partition → A or C; Else → Latency or Consistency. It captures that strong consistency costs latency even when nothing is broken.

Q5. Strong vs eventual consistency — when each?

Strong for money, inventory, locks — correctness over speed. Eventual for feeds, carts, DNS, counters — availability and latency over instant freshness.

Q6. How do you fix 'I posted but don't see it' with read replicas?

Read-your-writes: route the author's reads to the leader for a short window or use a version token / sticky replica.

Q7. Active-passive vs active-active?

Active-passive: one serves, standby takes over on failure (simpler, idle capacity, failover delay). Active-active: all serve traffic (better utilisation, faster recovery, harder state/conflict handling).

Q8. What is split brain and how do you prevent it?

Two nodes both act as primary after a partition. Prevent with quorum-based leader election (majority), fencing tokens, STONITH, or consensus systems like etcd/ZooKeeper.

Q9. How do you compute availability of a system with several components?

Multiply availabilities in series; for redundant parallel components use $1 - \prod(1 - A_i)$. Weakest link and correlated failures dominate in reality.

Q10. SLI vs SLO vs SLA?

SLI = measurement, SLO = internal target, SLA = customer contract with penalties (looser than the SLO). Error budget = $1 - SLO$.

Chapter 5 Cheat Sheet

◀ QUICK RECALL — Revise this in 2 minutes

- **Scalability** = handle growing load by adding resources. Find the bottleneck first; use percentiles (p99).
- **Vertical** = bigger box (simple, capped, SPOF). **Horizontal** = more boxes (needs **stateless** servers + LB + partitioned data). **Scale cube**: X clone, Y split by function, Z split by data.
- **Journey**: single box → separate DB → LB + stateless app → cache/CDN → read replicas → queues → shard → multi-region → microservices.
- **Availability** = $\text{uptime}/(\text{uptime}+\text{downtime})$. **99.9% ≈ 8.8 h/yr**, **99.99% ≈ 52 min/yr**, **99.999% ≈ 5 min/yr**.
- **Series**: multiply (gets worse). **Parallel**: $1 - \prod(1 - A)$ (gets better, if failures are independent). **Availability = MTBF/(MTBF+MTTR)** — recover fast.
- **SLI → SLO → SLA**; error budget = $1 - \text{SLO}$ (99.9% = 43 min/month).
- **Failover**: active-passive (hot/warm/cold standby, VIP takeover) vs active-active. Risks: data loss (async), **split brain** (quorum/fencing).
- **Replication**: sync (safe, slower) vs async (fast, lag, possible loss) vs semi-sync. Spread across AZs/regions; backups are separate from replication.
- **CAP**: P is mandatory → during a partition choose **C** (error/timeout) or **A** (stale answer). **PACELC**: else choose Latency vs Consistency.
- **Models**: strong > causal > read-your-writes / monotonic reads > eventual > weak. **Quorum**: $R + W > N$.
- **Use case rule**: money/inventory/locks → CP & strong; feeds/carts/DNS → AP & eventual; user's own edits → read-your-writes.

AMJESH
<https://singhamjesh.github.io/>